Module 11 · SW Architecture

Fundamentals of Software Architecture

Phase 2 begins. We zoom out from classes to whole systems.

Akın Kaldıroğlu · Lead Trainer

Where we are

From Small Scale to Large Scale

Phase 1 was design in the small — classes and functions. Phase 2 is design in the large — components, systems, and the qualities they must deliver.

Same forces

Cohesion & coupling still rule — now between components.

New stakes

Decisions here are costly to reverse.

New concern

Quality attributes: scalability, availability, security…

What you'll leave with

Learning Objectives

- Define software architecture and what makes a decision 'architectural'.
- Explain quality attributes — and why architecture is about trade-offs.
- Describe a system with three models: 4+1, C4, and the four dimensions.
- Place the styles we'll study — layered, hexagonal, distributed.

Recap, deeper

What Architecture Is

Architecture is the set of significant decisions about a system: its major components, how they relate, and the principles governing their evolution.

It's about

- Structure — the big parts & connectors
- Qualities — the '-ilities' the system must meet
- Constraints — the rules everyone follows

It's judged by

- Does it meet the quality attributes?
- Can the team build & evolve it?
- Are the costly decisions the right ones?

Section 01

Quality Attributes

Architecture exists to deliver the '-ilities'.

Two kinds of requirements

What vs How Well

Functional

- What the system does
- 'Members can borrow books'
- Features you can demo

Quality (non-functional)

- How well it does it
- 'Handles 10k users at 99.9% uptime'
- Where architecture lives or dies

Quality attributes

The Attributes That Shape Systems

Scalability

Grow with load and with the team.

Availability

Stay up despite failures.

Performance

Respond fast enough, under load.

Security

Protect data and access.

Maintainability

Cheap and safe to change.

Deployability

Ship often, with low risk.

Testability

Verify behavior at every level.

Observability

See what the system is doing.

The hard truth

Architecture Is Trade-Offs

You cannot maximize every attribute at once. Architecture is the art of choosing which qualities to favor — on purpose.

- More consistency often costs availability (and vice versa).
- More performance can cost simplicity and maintainability.
- More flexibility adds indirection and complexity.
- Name the attributes that matter most — then design for those.

Section 02

Architecture Models

One system, three models — each answering a different question.

Views

No Single Diagram Says It All

A system serves many stakeholders with different questions. We describe it through complementary views, each answering some of them.

- Developers ask: how is the code organized?
- Operators ask: what runs where, and how does it scale?
- Users & analysts ask: what does it do, end to end?
- One view can't answer all — so we use a small set.

Model

The 4+1 View Model

Logical

The functionality — key classes & responsibilities.

Process

Runtime — concurrency, processes, communication.

Development

Code organization — modules & packages.

Physical

Deployment — nodes, networks, infrastructure.

+1 Scenarios — use cases that tie the four views together and validate them.

Model

The C4 Model — Zoom Levels

C4 describes a system at four zoom levels, from the outside in — a practical, modern way to draw architecture.

- **Context — the system and the people & systems around it**
- **Container — apps, services & data stores that make it up**
- **Component — the major parts inside a container**
- **Code — classes & details (only where useful)**

Model

The Four Dimensions of Architecture

4+1 and C4 ask how to **draw** a system. Richards & Ford ask what an architecture is **made of** — four dimensions, each one decided by someone.

Logical components

The building blocks of functionality — and how they talk to each other.

Architecture characteristics

The “-ilities” the system must support — Section 01, an hour ago.

Architecture style

The overall shape: layered, hexagonal, event-driven, microservices.

Architecture decisions

The rules everyone builds by — long-lived, and costly to reverse.

Sources

Where These Models Come From

Three models, three primary sources. Go to these rather than to a blog summary — each says more than the one slide we gave it.

📖 **Architectural Blueprints — The “4+1” View Model** — *Philippe Kruchten · IEEE Software 12(6), Nov 1995, pp. 42–50*

The 4+1 paper, free to read → `arxiv.org/abs/2006.04975`

📖 **The C4 Model: Visualizing Software Architecture** — *Simon Brown · O’Reilly, 2026 — the model from the person who made it*

📖 **Software Architecture for Developers** — *Simon Brown · Leanpub — the architect’s role, and the balance with agility*

C4 model — diagrams that scale from context to code → `c4model.com`

📖 **Fundamentals of Software Architecture, 2nd ed.** — *Richards & Ford · O’Reilly, 2025 — ch. 1, the four dimensions*

Section 03

Styles & Patterns

Named, proven shapes for whole systems.

Vocabulary

Architectural Styles

An architectural style is a family of structures with shared principles — the large-scale cousin of a design pattern.

Layered

Organize by technical role — the classic n-tier. (Next session.)

Hexagonal / Clean

Domain at the center, infrastructure at the edges.

Distributed

Many services over a network — SOA & microservices.

Good practice

Record the Decisions

Architectural decisions are expensive to reverse — so capture them. An Architecture Decision Record (ADR) is a short note per significant choice.

An ADR captures

- The decision and its context
- The options considered
- The consequences & trade-offs

Why bother

- Future-you forgets the 'why'
- New joiners get the reasoning fast
- Revisit decisions honestly later

The AI angle

AI as an Architecture Thinking Partner

AI can enumerate options and trade-offs fast — but the significant, costly decisions are yours to make and own.

- Ask for candidate styles and their trade-offs against your quality goals.
- Have it draft C4 diagrams and ADRs from your description.
- You decide: it cannot feel the cost of getting an architecture wrong.

Remember these

Key Takeaways

- ✓ Architecture = the significant, costly-to-reverse decisions about a system.
- ✓ Quality attributes (the -ilities) are what architecture exists to deliver.
- ✓ You can't maximize every attribute — architecture is trade-offs.
- ✓ 4+1 and C4 describe a system; the four dimensions say what one is made of.
- ✓ Capture significant decisions in ADRs — the 'why' matters most.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Fundamentals of Software Architecture, 2nd ed.** Richards & Ford · O'Reilly, 2025

The course text for this phase: characteristics, styles, and architectural thinking.

📖 **Software Architecture in Practice, 4th ed.** Bass, Clements & Kazman · Addison-Wesley, 2021

Quality attributes done rigorously — scenarios, tactics, evaluation.

📖 **The C4 Model: Visualizing Software Architecture** Simon Brown · O'Reilly, 2026

The four zoom levels, in full — notation, worked examples and the tooling.

📖 **Just Enough Software Architecture** George Fairbanks · Marshall & Brainerd, 2010

A risk-driven model: how much architecture is enough, and when to stop.

The 4+1 paper, free to read → `arxiv.org/abs/2006.04975`

C4 model — diagrams that scale from context to code → `c4model.com`

Next session

What's Next

Module 12

Layered Architecture

with Akın Kaldıroğlu

The classic n-tier style — how it works, why it's everywhere, and the pain points that lead us to hexagonal and clean architectures.

Reference implementations

Ayvalık Bank — Two Architectures

The same banking system, built both ways. Open the two package trees side by side and compare how each names its parts.

Layered (LA)

Java `github.com/kaldiroglu/AyvalikBankLA-JAVA`

.NET `github.com/kaldiroglu/AyvalikBankLA-NET`

Python `github.com/kaldiroglu/AyvalikBankLA-Python`

IN THIS SESSION, LOOK AT

- `.../ayvalikbank: web, service, repository`

Hexagonal (HA)

Java `github.com/kaldiroglu/AyvalikBankHA-JAVA`

.NET `github.com/kaldiroglu/AyvalikBankHA-NET`

Python `github.com/kaldiroglu/AyvalikBankHA-Python`

IN THIS SESSION, LOOK AT

- `.../ayvalikbank: domain, application, adapter`

Questions?

Design the system for the qualities that matter.